T3: Backlink Monitor

Nov 10, 2026 · @Artur

1. Мета

Backlink Monitor щодня перевіряє куплені посилання й одразу повідомляє, якщо якесь зникло або перестало працювати на SEO. Проєкт робиться як портфоліо під вакансію Growica, де потрібна підтримка SEO-інструментів: проставлення посилань і CRM для аутричерів.

Проблема. Команда платить сайтам-донорам за посилання. Донори тихо видаляють їх, ставлять nofollow або закривають сторінку від індексації. Вручну перевіряти сотні посилань ніхто не встигає, тож гроші витрачаються даремно.

Користувачі. SEO-лід бачить загальну картину й гроші під ризиком. Аутричер отримує сигнал, що треба написати донору.

Поза межами MVP: перевірка індексації сторінки в Google (потрібен платний SERP API), авторизація, мультитенантність.

2. Вхідні дані

Посилання завантажуються CSV-файлом через дашборд або CLI (`npm run import links.csv`). Імпорт ідемпотентний: ключ — пара donor_url + target_url, повторне завантаження оновлює рядок, а не створює дубль.

Поле	Обов'язкове	Що це
donor_url	так	Сторінка, де стоїть посилання
target_url	так	Куди має вести посилання
anchor	ні	Очікуваний текст посилання
outreacher	ні	Хто домовився про посилання
price_usd	ні	Скільки заплачено
placed_at	ні	Дата розміщення (YYYY-MM-DD)

Приклад файлу:

```
donor_url,target_url,anchor,outreacher,price_usd,placed_at
https://blog-example.com/best-tips,https://my-site.com/,best casino
bonuses,Olha,150,2026-10-01
https://news-example.org/review,https://my-site.com/reviews,my-
site,Ivan,90,2026-10-12
```

Рядки з невалідним URL не валять імпорт: вони потрапляють у звіт помилок з номером рядка. Валідація — через Zod.

3. Перевірки і статуси

Кожне посилання перевіряється раз на 24 години та вручну кнопкою «Перевірити зараз». Перевірка = один HTTP GET сторінки-донора + розбір HTML через cheerio.

Що перевіряємо на сторінці:

1. HTTP-код та фінальний URL після редиректів (максимум 5).
2. Чи є `<a href>` на target_url. URL нормалізуються: без різниці http/https, www, слеша в кінці та utm-міток.
3. rel посилання: nofollow, sponsored, ugc.
4. Анкор: збіг з очікуваним (без урахування регістру та пробілів).
5. `<meta name="robots">` і заголовок X-Robots-Tag: noindex, nofollow.
6. `<link rel="canonical">`: чи веде на іншу сторінку.

Результат — один статус з пріоритетом зверху вниз та масив issues з деталями:

Статус	Коли	Алерт
`blocked`	403, 429, 503 після всіх ретраїв або знайдено капчу / Cloudflare challenge	ні, лише в дайджесті
`error`	таймаут, DNS, TLS, мережа після всіх ретраїв	ні, лише в дайджесті
`page_gone`	404 / 410	так
`lost`	сторінка 200, але посилання немає	так
`nofollow`	посилання є, але rel nofollow / sponsored / ugc	так

Статус	Коли	Алерт
`noindex`	сторінка закрита noindex або canonical на інший URL	так
`anchor_changed`	посилання живе, анкор інший	так, низький пріоритет
`ok`	все добре	ні

Головне правило: `blocked` і `error` ніколи не вважаються зникненням посилання. Алерт йде лише при ЗМІНІ статусу (був `ok` → став `lost`), щоб не спамити щодня. Для `lost` і `page_gone` алерт йде після 2 перевірок поспіль з інтервалом 1 година, щоб не реагувати на тимчасовий збій донора.

4. Краулер: проксі, ліміти, ретраї, капча

Це той розділ, який покриває вимогу вакансії «проксі, капча, ліміти запитів». Усі числа винесені в `.env`.

Параметр	Значення за замовчуванням	Навіщо
CONCURRENCY	10 запитів одночасно	Не перевантажити сервер і проксі
PER_DOMAIN_DELAY	1 запит / 5 с на домен	Донор з 20 посиланнями не забанить нас
TIMEOUT	15 с	Завислий сайт не блокує чергу
RETRIES	3, затримка 2 с → 8 с → 30 с + випадковий jitter	Тимчасові збої не стають фальшивими алертами
PROXY_LIST	файл proxies.txt, формат `http://user:pass@host:port`	Ротація IP
MAX_BODY	5 MB	Захист від величезних сторінок

Ретраї. Повторюємо лише те, що може полагодитися: таймаут, помилка мережі, 429, 5xx. 404 не повторюємо. Якщо є заголовок Retry-After, чекаємо стільки, скільки він каже.

Проксі. Кожен ретрай йде через інший проксі. Проксі з 3 помилками поспіль вимикається на 10 хвилин (circuit breaker). Якщо всі проксі мертві, черга стає на паузу і в Telegram йде один системний алерт. Без списку проксі працює напряму.

Заголовки. Реалістичний User-Agent з невеликого пулу та Accept-Language.

Капча і блокування. Капчу не розв'язуємо, а розпізнаємо: статус 403/503 + маркери в HTML (`cf-chl`, `challenge-platform`, `g-recaptcha`, `hcaptcha`, «Just a moment»). Результат — статус `blocked`. Наступний крок після MVP: fallback через Playwright для сайтів, що рендерять посилання JS-ом.

5. Сповіщення і дашборд

Telegram-бот (grammY або прямий Bot API) шле два типи повідомлень:

- Миттєвий алерт при зміні статусу: « ❗ Посилання зникло: blog-example.com/best-tips → my-site.com. Аутричер: Olha. Заплачено: $150» + лінк на картку в дашборді.
- Щоденний дайджест о 10:00: скільки ok / проблемних / blocked, сума $ під ризиком.
- Команди: `/status` (коротка сводка), `/check <url>` (перевірити одне посилання зараз).

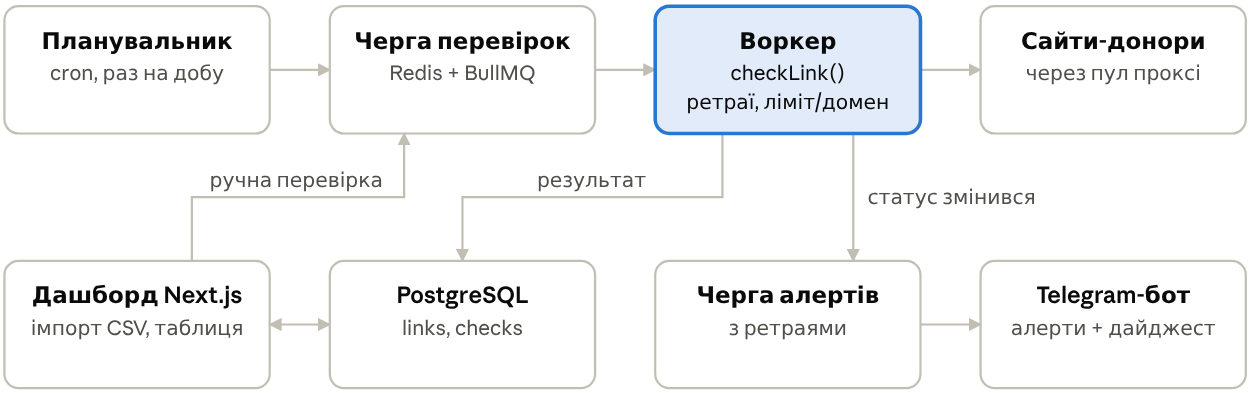
Дашборд (Next.js, без авторизації в MVP, або з одним паролем через env):

- Верхній рядок цифр: усього посилань, ok, проблемні, blocked, **$ під ризиком**.
- Таблиця посилань: фільтр за статусом і аутричером, сортування за ціною, кнопка «Перевірити зараз».
- Картка посилання: історія перевірок (дата, HTTP-код, статус, час відповіді, проксі).
- Звіт по аутричерах: скільки посилань, % живих, $ втрачено.
- Сторінка імпорту CSV зі звітом помилок.

6. Архітектура, стек, БД

Один репозиторій, два процеси: веб (дашборд + API) і воркер (черги + бот). Воркер і веб спілкуються лише через Postgres і Redis.

Алерт іде лише тоді, коли воркер бачить зміну статусу



архітектура · 8 компонентів

Планувальник або кнопка в дашборді кладуть джоби в чергу; воркер перевіряє донора, пише результат у БД і лише при зміні статусу ставить алерт у власну чергу.

Частина	Технологія
Мова	TypeScript, Node 22
HTTP + парсинг	undici (ProxyAgent) + cheerio
Черги і розклад	BullMQ + Redis
БД	PostgreSQL + Prisma
Валідація	Zod
Дашборд	Next.js + Tailwind + shadcn/ui
Бот	grammY
Тести	Vitest (HTML-фікстури для кожного статусу)
Локально	docker-compose: postgres + redis

Схема БД (Prisma):

```
model Link {
  id              String   @id @default(cuid())
  donorUrl        String
  targetUrl       String
  anchor          String?
  outreacher      String?
  priceUsd        Int?
  placedAt        DateTime?
  status          String   @default("pending")
  failStreak      Int      @default(0)
  lastCheckedAt   DateTime?
  statusChangedAt DateTime?
  checks          Check[]
  @@unique([donorUrl, targetUrl])
}

model Check {
  id         String   @id @default(cuid())
  link       Link     @relation(fields: [linkId], references: [id])
  linkId     String
  checkedAt  DateTime @default(now())
  httpCode   Int?
  status     String
  issues     Json     // ["rel=nofollow", "anchor: 'x' != 'y'"]
  proxy      String?
  durationMs Int?
  @@index([linkId, checkedAt])
}
```

7. Failure modes (де може зламатися)

Ця таблиця йде в README як є: вона показує «архітектурне мислення» з вакансії.

Що ламається	Наслідок без захисту	Як оброблено
Донор віддає капчу або 403	Фальшивий алерт «посилання зникло»	Статус `blocked`, без алерту
Донор ліг на годину	Алерт і паніка в команді	Ретраї + алерт лише після 2 перевірок поспіль

Що ламається	Наслідок без захисту	Як оброблено
Проксі помер	Усі перевірки падають	Circuit breaker, інший проксі на ретраї
Усі проксі мертві	Сотні `error` за хвилину	Пауза черги + один системний алерт
20 посилань на одному домені	Донор банить IP	Ліміт на домен (груповий rate limit BullMQ)
Посилання рендериться JS-ом	`lost`, хоча воно є	Відоме обмеження; план — Playwright fallback
URL в різних форматах (www, слеш, utm)	`lost`, хоча воно є	Нормалізація URL + unit-тести
Воркер впав посеред прогону	Частина посилань не перевірена	Джоби в Redis виживають, воркер дороблює після рестарту
Cron спрацював двічі	Дублі джобів і алертів	jobId = `${linkId}:${дата}`, BullMQ не додасть дубль
Telegram API недоступний	Алерт загублено	Алерти — окрема черга з ретраями
Сторінка 50 MB або не HTML	Воркер з'їдає пам'ять	Ліміт 5 MB + перевірка Content-Type

8. План на день і критерії готовності

Порядок роботи в Claude Code, приблизно 8 годин. Кожен крок — окремий коміт.

- [ ] Створити репо, `CLAUDE.md` з цим ТЗ, docker-compose (Postgres + Redis), Prisma-схема (~45 хв)
- [ ] Імпорт CSV з Zod-валідацією (~45 хв)
- [ ] `checkLink()`: fetch + cheerio + нормалізація URL + визначення статусу, unit-тести на HTML-фікстурах (~1.5 год)
- [ ] Ретраї, проксі-пул, детект капчі (~1 год)
- [ ] BullMQ: черга перевірок, ліміт на домен, щоденний розклад (~1 год)
- [ ] Telegram: алерти при зміні статусу + дайджест (~45 хв)

- [ ] Дашборд: цифри, таблиця, картка посилання (~1.5 год)
- [ ] README + демо GIF + деплой (Railway або Render) (~45 хв)

Готово, коли:

1. Імпортуєш CSV з 10 посилань → усі 10 перевірені та мають статус.
2. Видаляєш посилання на тестовій сторінці (свій сайт на Vercel) → приходить алерт у Telegram.
3. Сторінка з капчею показує `blocked`, а не `lost`.
4. Падіння воркера посеред прогону не губить перевірки.
5. `npm test` зелений, дашборд відкривається за публічним лінком.

README має містити: проблему в одному абзаці, GIF, схему архітектури, таблицю статусів, таблицю Failure modes, запуск за 3 команди та розділ «How I built it with Claude Code» (як ставив задачі, що виправляв руками).